

BEACONHOUSE NATIONAL UNIVERSITY



LAB 5 | WEEK 5

Name: Saad Mughal

BNU ID: F2023-009

Due Date: 14 Mar, 2025

Lab Manual

For

ADMS(SEC_B)

Course Instructor	Ms. Talia Noreen
Lab Instructor	Mr. Ahsan Atiq
Semester	Spring 2025

1. What is the maximum, minimum, average and standard deviation of ages of the users?
(Search Standard Dev function)

The screenshot shows MySQL Workbench with the following SQL queries and results:

```

76 (4, 'Data Science'),
77 (5, 'Music');
78
79 CREATE TABLE UserInterests (
80
81 INSERT INTO UserInterests (user_id, interest_id) VALUES
82 (1, 1),
83 (1, 3),
84 (2, 2),
85 (2, 4),
86 (3, 5),
87 (4, 3),
88 (5, 1),
89 (5, 4)
90
91 -- 1. What is the maximum, minimum, average and standard deviation of ages of the users? (Search Standard Dev function)
92
93 SELECT
94     max(age) as max_age,
95     min(age) as min_age,
96     avg(age) as avg_age,
97     stddev(age) as stddev_age
98 from Users;

```

Result Grid:

max_age	min_age	avg_age	stddev_age
28	22	25.0000	4.62788724238731

Action Output:

#	Time	Action	Message	Duration / Fetch
13	21:27:04	CREATE TABLE UserInterests (user_id INT, interest_id INT, FOREIGN KEY (user_id) REFERENCES Users(user_id), FOREIGN KEY (interest_id) REFERENCES UserInterests(interest_id))	0 row(s) affected	0.031 sec
14	21:27:06	INSERT INTO UserInterests (user_id, interest_id) VALUES (1, 1), (1, 3), (2, 2), (2, 4), (3, 5), (4, 3), (5, 1), (5, 4)	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.015 sec
15	21:35:10	SELECT max(age) as max_age, min(age) as min_age, avg(age) as avg_age, stddev(age) as stddev_age from Users LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

2. Give the name of the user who has the highest number of followers.

The screenshot shows MySQL Workbench with the following SQL queries and results:

```

92 (4, 1),
93 (5, 1),
94 (5, 4);
95
96 -- 1. What is the maximum, minimum, average and standard deviation of ages of the users? (Search Standard Dev function)
97
98 SELECT
99     max(age) as max_age,
100     min(age) as min_age,
101     avg(age) as avg_age,
102     stddev(age) as stddev_age
103 from Users;
104
105 -- 2. Give the name of the user who has the highest number of followers.
106
107 select u.username
108 from Users u
109 join (
110     select user_id, count(follower_id) as follower_count
111     from Followers
112     group by user_id
113     order by follower_count desc
114     limit 1
115 ) sub on u.user_id = sub.user_id;

```

Result Grid:

username
Alice

Action Output:

#	Time	Action	Message	Duration / Fetch
15	21:35:10	SELECT max(age) as max_age, min(age) as min_age, avg(age) as avg_age, stddev(age) as stddev_age from Users LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
16	21:38:27	select u.username from Users u join (select user_id, count(follower_id) as follower_count from Followers group by user_id order by follower_count desc limit 1) sub on u.user_id = sub.user_id;	1 row(s) returned	0.015 sec / 0.000 sec
17	21:38:41	select u.username from Users u join (select user_id, count(follower_id) as follower_count from Followers group by user_id order by follower_count desc limit 1) sub on u.user_id = sub.user_id;	1 row(s) returned	0.000 sec / 0.000 sec

3. Give name of the user who has second highest followers.

The screenshot shows the MySQL Workbench interface with the 'socialmedia' schema selected. The SQL editor contains two queries. The first query finds the user with the highest number of followers, and the second query finds the user with the second highest number of followers. The results are displayed in the 'Result Grid' at the bottom.

```
-- 2. Give the name of the user who has the highest number of followers.
104 select u.username
105 from Users u
106 join (
107     select user_id, count(follower_id) as follower_count
108     from Followers
109     group by user_id
110     order by follower_count desc
111     limit 1
112 ) sub on u.user_id = sub.user_id;

-- 3. Give name of the user who has second highest followers.
115 SELECT u.username
116 FROM Users u
117 JOIN (
118     SELECT user_id, COUNT(follower_id) AS follower_count
119     FROM Followers
120     GROUP BY user_id
121     ORDER BY follower_count DESC
122     LIMIT 1 OFFSET 1
123 ) sub ON u.user_id = sub.user_id;
```

Result 4

username
Bob

Result 5

username
Bob

Output

Time	Action	Message	Duration / Fetch
16 21:38:27	select u.username from Users u join (select user_id, count(follower_id) as follower_count from Followers group by user_id order by follower_count...	1 row(s) returned	0.016 sec / 0.000 sec
17 21:38:41	select u.username from Users u join (select user_id, count(follower_id) as follower_count from Followers group by user_id order by follower_count...	1 row(s) returned	0.000 sec / 0.000 sec
18 21:55:45	SELECT u.username FROM Users u JOIN (SELECT user_id, COUNT(follower_id) AS follower_count FROM Followers GROUP BY user_id O...	1 row(s) returned	0.000 sec / 0.000 sec

4. List names of all the users who have never tweeted.

The screenshot shows the MySQL Workbench interface with the 'socialmedia' schema selected. The SQL editor contains two queries. The first query finds the user with the second highest number of followers, and the second query finds the names of all the users who have never tweeted. The results are displayed in the 'Result Grid' at the bottom.

```
-- 3. Give name of the user who has second highest followers.
115 SELECT u.username
116 FROM Users u
117 JOIN (
118     SELECT user_id, COUNT(follower_id) AS follower_count
119     FROM Followers
120     GROUP BY user_id
121     ORDER BY follower_count DESC
122     LIMIT 1 OFFSET 1
123 ) sub ON u.user_id = sub.user_id;

-- 4. List names of all the users who have never tweeted.
126 SELECT username
127 FROM Users
128 WHERE user_id NOT IN (SELECT DISTINCT user_id FROM Tweets);
```

Result 4

username
Bob

Result 5

username
Charlie

Output

Time	Action	Message	Duration / Fetch
17 21:38:41	select u.username from Users u join (select user_id, count(follower_id) as follower_count from Followers group by user_id order by follower_count...	1 row(s) returned	0.000 sec / 0.000 sec
18 21:55:45	SELECT u.username FROM Users u JOIN (SELECT user_id, COUNT(follower_id) AS follower_count FROM Followers GROUP BY user_id O...	1 row(s) returned	0.000 sec / 0.000 sec
19 21:56:43	SELECT username FROM Users WHERE user_id NOT IN (SELECT DISTINCT user_id FROM Tweets) LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

5. List all the hashtags and usernames and number of times that user used that hashtag.

The screenshot shows MySQL Workbench with a SQL editor containing three queries. The first query finds the user with the second highest number of followers. The second query lists users who have never tweeted. The third query lists all hashtags and usernames along with the number of times each user used each hashtag.

```
-- 3. Give name of the user who has second highest followers.
SELECT u.username
FROM Users u
JOIN (
  SELECT user_id, COUNT(follower_id) AS follower_count
  FROM Followers
  GROUP BY user_id
  ORDER BY follower_count DESC
  LIMIT 1 OFFSET 1
) sub ON u.user_id = sub.user_id;

-- 4. List names of all the users who have never tweeted.
SELECT username
FROM Users
WHERE user_id NOT IN (SELECT DISTINCT user_id FROM Tweets);

-- 5. List all the hashtags and usernames and number of times that user used that hashtag.
SELECT u.username, h.hashtag, COUNT(*) AS usage_count
FROM Tweets t
JOIN Hashtags h ON t.tweet_id = h.tweet_id
JOIN Users u ON t.user_id = u.user_id
GROUP BY u.username, h.hashtag;
```

The results for the third query are shown in a table:

username	hashtag	usage_count
Alice	#AI	1
Bob	#Census	1
Bob	#Tech	1
David	#Code	1
Emma	#AI	1

6. Find the users who have never used the hashtag #Census.

The screenshot shows MySQL Workbench with a SQL editor containing three queries. The first query lists users who have never tweeted. The second query lists all hashtags and usernames along with the number of times each user used each hashtag. The third query finds users who have never used the hashtag #Census.

```
-- 4. List names of all the users who have never tweeted.
SELECT username
FROM Users
WHERE user_id NOT IN (SELECT DISTINCT user_id FROM Tweets);

-- 5. List all the hashtags and usernames and number of times that user used that hashtag.
SELECT u.username, h.hashtag, COUNT(*) AS usage_count
FROM Tweets t
JOIN Hashtags h ON t.tweet_id = h.tweet_id
JOIN Users u ON t.user_id = u.user_id
GROUP BY u.username, h.hashtag;

-- 6. Find the users who have never used the hashtag #Census.
SELECT username
FROM Users
WHERE user_id NOT IN (
  SELECT DISTINCT t.user_id
  FROM Tweets t
  JOIN Hashtags h ON t.tweet_id = h.tweet_id
  WHERE h.hashtag = '#Census'
);
```

The results for the third query are shown in a table:

username
Alice
Charlie
David
Emma

7. List all the usernames that have never been followed. Using Set operation.

The screenshot shows the MySQL Workbench interface with the following SQL queries and results:

```
-- 5. List all the hashtags and usernames and number of times that user used that hashtag.
131 SELECT u.username, h.hashtag, COUNT(*) AS usage_count
132 FROM Tweets t
133 JOIN Hashtags h ON t.tweet_id = h.tweet_id
134 JOIN Users u ON t.user_id = u.user_id
135 GROUP BY u.username, h.hashtag;

-- 6. Find the users who have never used the hashtag #Census.
138 SELECT username
139 FROM Users
140 WHERE user_id NOT IN (
141     SELECT DISTINCT t.user_id
142     FROM Tweets t
143     JOIN Hashtags h ON t.tweet_id = h.tweet_id
144     WHERE h.hashtag = '#Census'
145 );

-- 7. List all the usernames that have never been followed. Using Set operation.
148 SELECT username
149 FROM Users
150 WHERE user_id NOT IN (SELECT DISTINCT follower_id FROM Followers);
```

The results pane shows the output of the last query:

username
...

The bottom pane shows the execution log with the following entries:

Time	Action	Message	Duration / Pctch
20 21:57:12	SELECT username, h.hashtag, COUNT(*) AS usage_count FROM Tweets t JOIN Hashtags h ON t.tweet_id = h.tweet_id JOIN Users u ON t.user_id = u.user_id GROUP BY u.username, h.hashtag;	5 rows(s) returned	0.000 sec / 0.000 sec
21 21:58:24	SELECT username FROM Users WHERE user_id NOT IN (SELECT DISTINCT t.user_id FROM Tweets t JOIN Hashtags h ON t.tweet_id = h.tweet_id WHERE h.hashtag = '#Census');	4 rows(s) returned	0.000 sec / 0.000 sec
22 21:58:42	SELECT username FROM Users WHERE user_id NOT IN (SELECT DISTINCT follower_id FROM Followers) LIMIT 0, 1000	0 rows(s) returned	0.000 sec / 0.000 sec

8. List all the usernames that have never been followed. Using Exist Clause.

The screenshot shows the MySQL Workbench interface with the following SQL queries and results:

```
-- 6. Find the users who have never used the hashtag #Census.
137 SELECT username
138 FROM Users
139 WHERE user_id NOT IN (
140     SELECT DISTINCT t.user_id
141     FROM Tweets t
142     JOIN Hashtags h ON t.tweet_id = h.tweet_id
143     WHERE h.hashtag = '#Census'
144 );

-- 7. List all the usernames that have never been followed. Using Set operation.
148 SELECT username
149 FROM Users
150 WHERE user_id NOT IN (SELECT DISTINCT follower_id FROM Followers);

-- 8. List all the usernames that have never been followed. Using Exist Clause.
153 SELECT username
154 FROM Users u
155 WHERE NOT EXISTS (
156     SELECT 1 FROM Followers f WHERE f.follower_id = u.user_id
157 );
```

The results pane shows the output of the last query:

username
...

The bottom pane shows the execution log with the following entries:

Time	Action	Message	Duration / Pctch
21 21:58:24	SELECT username FROM Users WHERE user_id NOT IN (SELECT DISTINCT t.user_id FROM Tweets t JOIN Hashtags h ON t.tweet_id = h.tweet_id WHERE h.hashtag = '#Census');	4 rows(s) returned	0.000 sec / 0.000 sec
22 21:58:42	SELECT username FROM Users WHERE user_id NOT IN (SELECT DISTINCT follower_id FROM Followers) LIMIT 0, 1000	0 rows(s) returned	0.000 sec / 0.000 sec
23 21:59:10	SELECT username FROM Users u WHERE NOT EXISTS (SELECT 1 FROM Followers f WHERE f.follower_id = u.user_id LIMIT 0, 1000	0 rows(s) returned	0.000 sec / 0.000 sec

9. Find the most common interest of users. (The interest with the largest number of users). Also find the least common interest.

The screenshot shows the MySQL Workbench interface with a query editor and a results grid. The query is designed to find the most common interest of users by joining the UserInterests table with itself and counting the occurrences of each interest.

```

-- 8. List all the usernames that have never been followed. Using Exist Clause.
153 SELECT username
154 FROM Users u
155 WHERE NOT EXISTS (
156     SELECT 1 FROM Followers f WHERE f.follower_id = u.user_id
157 )
158
-- 9. Find the most common Interest (Interest with the maximum number of users). Also find the least common interest.
159
160 SELECT UI.Interest_Id, I.description, COUNT(UI.user_id) AS MaxCount
161 FROM UserInterests UI
162 JOIN Interests I ON UI.Interest_Id = I.Interest_Id
163 GROUP BY UI.Interest_Id, I.description
164 HAVING COUNT(UI.user_id) = (
165     SELECT MAX(NumInterests)
166     FROM (
167         SELECT COUNT(user_id) AS NumInterests
168         FROM UserInterests
169         GROUP BY Interest_Id
170     ) AS Inters
171 ) AS Inters
172
173

```

The results grid shows the following data:

Interest_Id	description	MaxCount
1	AI	2
3	Coding	2
4	Data Science	2

The Action Output pane shows the execution of the query, indicating that 4 rows were returned.

The screenshot shows the MySQL Workbench interface with a query editor and a results grid. The query is designed to find the least common interest of users by joining the UserInterests table with itself and counting the occurrences of each interest.

```

166 HAVING COUNT(UI.user_id) = (
167     SELECT MAX(NumInterests)
168     FROM (
169         SELECT COUNT(user_id) AS NumInterests
170         FROM UserInterests
171         GROUP BY Interest_Id
172     ) AS Inters
173 )
174
-- Find the least common Interest (Interest with the minimum number of users)
175
176 SELECT UI.Interest_Id, I.description, COUNT(UI.user_id) AS MinCount
177 FROM UserInterests UI
178 JOIN Interests I ON UI.Interest_Id = I.Interest_Id
179 GROUP BY UI.Interest_Id, I.description
180 HAVING COUNT(UI.user_id) = (
181     SELECT MIN(NumInterests)
182     FROM (
183         SELECT COUNT(user_id) AS NumInterests
184         FROM UserInterests
185         GROUP BY Interest_Id
186     ) AS Inters
187 )
188

```

The results grid shows the following data:

Interest_Id	description	MinCount
2	Machine Learning	1
5	Music	1

The Action Output pane shows the execution of the query, indicating that 2 rows were returned.

10. Show total tweets per country. The result should be in order of country name.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'socialmedia' selected. The main editor contains a SQL query with line numbers 145 to 167. The query includes a comment about showing total tweets per country. The 'Result Grid' at the bottom shows the results of the query.

```
145 WHERE h.hashtag = 'BCensus'
146
147
148
149 -- 7. List all the usernames that have never been followed. Using Set operation.
150 SELECT username
151 FROM Users
152 WHERE user_id NOT IN (SELECT DISTINCT follower_id FROM Followers);
153
154 -- 8. List all the usernames that have never been followed. Using Exist Clause.
155 SELECT username
156 FROM Users u
157 WHERE NOT EXISTS (
158     SELECT 1 FROM Followers f WHERE f.follower_id = u.user_id
159 );
160
161 -- 9. Find the most common interest of users. (The interest with the largest number of users). Also find the least common interest.
162
163 -- 10. Show total tweets per country. The result should be in order of country name.
164 SELECT country, SUM(number_of_tweets) AS total_tweets
165 FROM Users
166 GROUP BY country
167 ORDER BY country;
```

country	total_tweets
Canada	2
Pakistan	3
UK	7
USA	5

The 'Output' pane shows the execution details of the query, including the time taken (22:00:14) and the number of rows returned (4 rows).

11. List names of all the users whose number of tweets is more than the average number of tweets per user.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'socialmedia' selected. The main editor contains a SQL query with line numbers 150 to 171. The query includes a comment about listing users whose number of tweets is more than the average. The 'Result Grid' at the bottom shows the results of the query.

```
150 FROM Users
151 WHERE user_id NOT IN (SELECT DISTINCT follower_id FROM Followers);
152
153 -- 8. List all the usernames that have never been followed. Using Exist Clause.
154 SELECT username
155 FROM Users u
156 WHERE NOT EXISTS (
157     SELECT 1 FROM Followers f WHERE f.follower_id = u.user_id
158 );
159
160 -- 9. Find the most common interest of users. (The interest with the largest number of users). Also find the least common interest.
161
162 -- 10. Show total tweets per country. The result should be in order of country name.
163 SELECT country, SUM(number_of_tweets) AS total_tweets
164 FROM Users
165 GROUP BY country
166 ORDER BY country;
167
168 -- 11. List names of all the users whose number of tweets is more than the average number of tweets per user.
169 SELECT username
170 FROM Users
171 WHERE number_of_tweets > (SELECT AVG(number_of_tweets) FROM Users);
```

username
Alice
David

The 'Output' pane shows the execution details of the query, including the time taken (22:00:19) and the number of rows returned (2 rows).

12. Give the name of the users who have at least one follower from Pakistan.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'socialmedia' selected. The main editor contains a SQL query with comments for exercises 9 through 12. The query for exercise 12 is: `SELECT DISTINCT u.username FROM Users u JOIN Followers f ON u.user_id = f.user_id WHERE f.user.country = 'Pakistan';`. The 'Result Grid' shows the output of this query, listing the usernames 'Alice', 'David', 'Bob', and 'Charlie'.

```
157 SELECT 1 FROM Followers f WHERE f.follower_id = u.user_id
158
159
160 -- 9. Find the most common interest of users. (The interest with the largest number of users). Also find the least common interest.
161
162 -- 10. Show total tweets per country. The result should be in order of country name.
163 SELECT country, SUM(number_of_tweets) AS total_tweets
164 FROM Users
165 GROUP BY country;
166
167 -- 11. List names of all the users whose number of tweets is more than the average number of tweets per user.
168 SELECT username
169 FROM Users
170 WHERE number_of_tweets > (SELECT AVG(number_of_tweets) FROM Users);
171
172 -- 12. Give the name of the users who have at least one follower from Pakistan.
173 SELECT DISTINCT u.username
174 FROM Users u
175 JOIN Followers f ON u.user_id = f.user_id
176 JOIN Users f_user ON f.follower_id = f_user.user_id
177 WHERE f_user.country = 'Pakistan';
178
```

username
Alice
David
Bob
Charlie

Result 13 -> Output

Time	Action	Message	Duration / Fetch
28 22:00:37	SELECT country, SUM(number_of_tweets) AS total_tweets FROM Users GROUP BY country ORDER BY country LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
29 22:01:01	SELECT username FROM Users WHERE number_of_tweets > (SELECT AVG(number_of_tweets) FROM Users) LIMIT 0, 1000	2 row(s) returned	0.016 sec / 0.000 sec
30 22:01:22	SELECT DISTINCT u.username FROM Users u JOIN Followers f ON u.user_id = f.user_id JOIN Users f_user ON f.follower_id = f_user.user_id WHERE f_user.country = 'Pakistan'	4 row(s) returned	0.000 sec / 0.000 sec

13. Show the interest ID and description of interest with the most number of users.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'socialmedia' selected. The main editor contains a SQL query with comments for exercises 11 through 13. The query for exercise 13 is: `SELECT i.interest_id, i.description, COUNT(ui.user_id) AS user_count FROM UserInterests ui JOIN Interests i ON ui.interest_id = i.interest_id GROUP BY i.interest_id, i.description ORDER BY user_count DESC LIMIT 1;`. The 'Result Grid' shows the output of this query, displaying the interest ID '1' and description 'AI' with a user count of 2.

```
165 GROUP BY country;
166 ORDER BY country;
167
168 -- 11. List names of all the users whose number of tweets is more than the average number of tweets per user.
169 SELECT username
170 FROM Users
171 WHERE number_of_tweets > (SELECT AVG(number_of_tweets) FROM Users);
172
173 -- 12. Give the name of the users who have at least one follower from Pakistan.
174 SELECT DISTINCT u.username
175 FROM Users u
176 JOIN Followers f ON u.user_id = f.user_id
177 JOIN Users f_user ON f.follower_id = f_user.user_id
178 WHERE f_user.country = 'Pakistan';
179
180 -- 13. Show the interest ID and description of interest with the most number of users.
181 SELECT i.interest_id, i.description, COUNT(ui.user_id) AS user_count
182 FROM UserInterests ui
183 JOIN Interests i ON ui.interest_id = i.interest_id
184 GROUP BY i.interest_id, i.description
185 ORDER BY user_count DESC
186 LIMIT 1;
```

interest_id	description	user_count
1	AI	2

Result 14 -> Output

Time	Action	Message	Duration / Fetch
29 22:01:01	SELECT username FROM Users WHERE number_of_tweets > (SELECT AVG(number_of_tweets) FROM Users) LIMIT 0, 1000	2 row(s) returned	0.016 sec / 0.000 sec
30 22:01:22	SELECT DISTINCT u.username FROM Users u JOIN Followers f ON u.user_id = f.user_id JOIN Users f_user ON f.follower_id = f_user.user_id WHERE f_user.country = 'Pakistan'	4 row(s) returned	0.000 sec / 0.000 sec
31 22:01:46	SELECT interest_id, i.description, COUNT(ui.user_id) AS user_count FROM UserInterests ui JOIN Interests i ON ui.interest_id = i.interest_id GROUP BY i.interest_id, i.description ORDER BY user_count DESC LIMIT 1	1 row(s) returned	0.000 sec / 0.000 sec